Impact of Tokenization on Language Models: An Analysis for Turkish

CAGRI TORAMAN, EYUP HALIT YILMAZ, FURKAN ŞAHİNUÇ, and OGUZHAN OZCELIK, Aselsan Research Center, Turkey

Tokenization is an important text preprocessing step to prepare input tokens for deep language models. WordPiece and BPE are de facto methods employed by important models, such as BERT and GPT. However, the impact of tokenization can be different for morphologically rich languages, such as Turkic languages, in which many words can be generated by adding prefixes and suffixes. We compare five tokenizers at different granularity levels, that is, their outputs vary from the smallest pieces of characters to the surface form of words, including a Morphological-level tokenizer. We train these tokenizers and pretrain medium-sized language models using the RoBERTa pretraining procedure on the Turkish split of the OSCAR corpus. We then fine-tune our models on six downstream tasks. Our experiments, supported by statistical tests, reveal that the morphological-level tokenizer delivers a challenging performance with de facto tokenizers. Furthermore, we find that increasing the vocabulary size improves the performance of Morphological- and Word-level tokenizers more than that of de facto tokenizers. The ratio of the number of vocabulary parameters to the total number of model parameters can be empirically chosen as 20% for de facto tokenizers and 40% for other tokenizers to obtain a reasonable trade-off between model size and performance.

CCS Concepts: • **Computing methodologies** → **Natural language processing**; *Language resources*; Phonology/morphology; Modeling methodologies;

Additional Key Words and Phrases: Language model, morphological analysis, tokenization, vocabulary size

ACM Reference format:
Cagri Toraman, Eyup Halit Yilmaz, Furkan Şahinuç, and Oguzhan Ozcelik. 2023. Impact of Tokenization on Language Models: An Analysis for Turkish. *ACM Trans. Asian Low-Resour. Lang. Inf. Process.* 22, 4, Article 116 (March 2023), 21 pages.
https://doi.org/10.1145/3578707

1 INTRODUCTION

Deep language models gained popularity with the introduction of masked language modeling based on the Transformer architecture [Vaswani et al. 2017] to pretrain a general purpose language understanding with BERT [Devlin et al. 2019] and its variants. The language models are then able to transfer the pretrained knowledge to downstream tasks, such as Sentiment Analysis and Named Entity Recognition. Such large models provide impressive results on the performance of many downstream tasks, not only in natural language processing [Devlin et al. 2019] but also in many other research areas, such as search [Yates et al. 2021] and recommendation [Sun et al. 2019].

Authors' address: C. Toraman, E. H. Yilmaz, F. Şahinuç, and O. Ozcelik, Aselsan Research Center, Ankara, Turkey, Yenimahalle, Mehmet Akif Ersoy Mahallesi, Istiklal Marsi Caddesi, 06200, emails: {ctoraman, ehyilmaz, fsahinuc, ogozcelik}@aselsan.com.tr.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2375-4699/2023/03-ART116 $15.00
https://doi.org/10.1145/3578707

Table 1. Word Examples for Morphological Analysis in Turkish

Word	Morphological Analysis	Translation
verdim	ver - di - m	I gave
vermedim	ver - me - di - m	I did not give
veremedim	ver - e - me - di - m	I could not give
kadının	kadı - nın	judge's
kadının	kadın - ın	woman's
evde	ev - de	in house
evden	ev - den	from house
biçare	bi - çare	helpless

Different colors indicate the mapping of suffixes to the translations in English.

Tokenization is an important text preprocessing step for deep language models. Conventional word embeddings, such as word2vec [Mikolov et al. 2013], generally use vocabularies consisting of the surface forms of words. On the other hand, deep language models employ more efficient tokenization algorithms in which input text is split into smaller pieces so that out-of-vocabulary words can still be processed. Language models can also benefit from the tokens that represent basic semantic units to better comprehend text semantics.

Transformer-based language models generally employ two de facto tokenization algorithms: WordPiece [Schuster and Nakajima 2012] and Byte Pair Encoding (BPE) [Sennrich et al. 2016]. For instance, BERT [Devlin et al. 2019] uses WordPiece, whereas GPT-2 [Radford et al. 2019] employs BPE. Large language models are first pretrained for English; successor pretrained models in low-resource languages thereby employ the same tokenizers [Schweter 2020]. However, the impact of tokenization algorithms can be different for low-resource languages, such as agglutinative Turkic and Uralic languages, in which words can have prefixes and suffixes. Moreover, the impact of different tokenization methods, including token representation in different levels from character level to word level, is not examined in detail for low-resource languages, specifically for Turkish.

In Table 1, we present a set of morphological analysis examples for possible challenges in Turkish. We list the parsed versions of three words in Turkish along with their English translation: "verdim" (translated as "I gave"), "vermedim" (translated as "I did not give"), and "veremedim" (translated as "I could not give"). Each example includes a different number of suffixes, and suffixes can be responsible for different meanings. For instance, parsing the word "veremedim" results in "ver-e-me-di-m," including four suffixes in a single word. A Morphological-level tokenizer can output five tokens in this case, providing the model with a better understanding of word semantics. An example benefit is that the language model would relate the suffix "-me" to negation, similar to the word "not" in English. Similarly, the suffix "-di" provides the past tense. More interestingly, adding the suffix "-e" to "ver-me-di-m" results in "ver-e-me-di-m" which changes the meaning from "I did not give" to "I could not give."

In Turkish, two words can be syntactically identical due to appending different suffixes while exposing different meanings according to the context. In such cases, readers need to infer the implied meaning from the context. Note that this case is different from homonymy, in which words have the same pronunciation but different spellings and meanings. In Table 1, we provide some examples in Turkish. The word "kadının" can have different meanings in its context. Moreover, a single-letter difference in suffixes can change the meaning of the word. In the examples of "evde" and "evden," the suffix "-de" means the preposition "in" and the suffix "-den" means the preposition "from." There are also prefixes in the Turkish language. For instance, the prefix "bi-" in the word "biçare" gives the meaning of deprivation in Table 1. Considering the cases and examples

mentioned in Table 1, tokenization and morphological analysis have significant importance in Turkish language processing.

The number of unique tokens used in training deep language models is referred to as the *vocabulary size*. Any evaluation data would be split into tokens by using the selected tokenization algorithm according to the trained vocabulary. There is a likelihood of observing out-of-vocabulary or unknown tokens, that is, some tokens in the evaluation data can be missing in the trained vocabulary. The problem with unknown tokens is that they are mapped to the same embedding without semantic context, resulting in possible performance loss. As the vocabulary size increases, the likelihood of getting such unknown tokens is decreased, since the vocabulary would capture more instances of tokens. On the other hand, the model becomes less efficient in terms of its size, that is, the memory requirement would increase and the model would become more costly to train. This results in a trade-off between model size and performance in terms of vocabulary size.

The objectives of this paper are to provide an analysis of important tokenization algorithms in the context of Transformer-based language models, to contribute to the research on Turkish language processing by providing the pretrained models online, and to give an idea of the broader impact and ethical issues around this research topic, particularly the environmental cost of pretraining. The actors who would benefit from our study are the researchers studying Turkish language processing and the practitioners utilizing such models in the industry. In this study, we thereby examine the following research questions.

- **RQ-1**. What is the impact of different tokenization methods on the performance of Turkish language modeling and varying downstream tasks? For example, does a Morphological-level tokenization method provide benefits for Turkish language modeling?
- **RQ-2**. How does the model performance change in different tokenization methods when the vocabulary size is tuned for the trade-off between model size and performance?

In order to answer our research questions, we compare the performance of different tokenization methods for Turkish. We select five tokenizers at different granularity levels, that is, their outputs vary from the smallest pieces (characters) to the surface form (words), which are Character-level, BPE, WordPiece, Morphological-level, and Word-level tokenization, respectively. In order to evaluate their performances, we train a tokenizer for each method and pretrain medium language models using the RoBERTa [Liu et al. 2019] pretraining procedure on the Turkish split of the OSCAR [Ortiz Suárez et al. 2019] corpus, called **RoBERTa-TR-medium**.[1] We then evaluate the performance of our models by fine-tuning them on six downstream tasks: News Classification, Hate Speech Detection, Sentiment Analysis, Named Entity Recognition, Semantic Text Similarity (STS), and Natural Language Inference (NLI).

The **main contributions and practical implications** of this study can be summarized as follows.

- We analyze the impact of tokenizers, at different granularity levels from character level to word level, on varying downstream tasks for Turkish language models. We find that the Morphological-level tokenizer is competitive with de facto tokenizers, that is, BPE and WordPiece. Our experimental results, supported by statistical tests, can shed light on the role of tokenization in language modeling, specifically for morphologically rich languages.
- We show that increasing the vocabulary size improves the performances of Morphological- and Word-level tokenizers more than that of the de facto tokenizers, BPE and WordPiece. The ratio of the number of vocabulary parameters to the total number of model parameters can be empirically chosen as 20% for de facto tokenizers and 40% for others. This choice

[1]We publish our pretrained models with different tokenizers and vocabulary sizes at https://huggingface.co/ctoraman.

can result in more efficient use of computational resources, in which the majority of the parameters are allocated to the Transformer blocks instead of the vocabulary embeddings. Moreover, we believe that our experimental results on vocabulary size can provide empirical guidance for other researchers who work on pretraining deep language models.

- We compare our medium-based models with a state-of-the-art Turkish language model [Schweter 2020] that has the same model architecture with BERT-base [Devlin et al. 2019] and show that our approximately 3-times-smaller model can recover 97% of the performance of the larger one. Our language models are publicly available so that other researchers and practitioners can benefit from them in terms of more efficient model sizes and varying vocabulary sizes. This would reduce memory requirements and also provide responsible energy usage and a smaller carbon footprint in return. We discuss these aspects in Section 5 in detail along with other ethical concerns, including transparency and fairness.

The rest of the article is organized as follows. In Section 2, we provide a brief literature review of tokenization algorithms in general and tokenization in low-resource languages. We explain the details of tokenization methods, our pretrained model, and downstream tasks in Section 3. Our comparative experiments on different tokenizers and analysis of vocabulary size are given in Section 4. We then provide a short discussion on the ethical concerns and broader impact of our study in Section 5. We present our conclusions in Section 6.

2 RELATED WORK

2.1 Tokenization Algorithms

Since tokenization is one of the first steps in any Information Retrieval or Natural Language Processing system, the importance of using a tokenization algorithm is highlighted in early studies [Metke Jimenez et al. 2011]. The prevalent tokenization algorithms in the literature, Byte Pair Encoding (BPE) [Sennrich et al. 2016] and WordPiece [Schuster and Nakajima 2012], are of recent interest in language model pretraining research. Many outstanding studies in the literature focus on enhancing these subword tokenization methods. For example, Ding et al. [2019b] explore the impact of the number of BPE merges on the machine translation performance. Provilkov et al. [2020] propose a drop-out method for each merge step of BPE in order to break the deterministic nature of BPE, which provides a performance improvement in machine translation.

BPE is found to be suboptimal for language pretraining [Bostrom and Durrett 2020] as it does not effectively utilize the vocabulary space. Nayak et al. [2020] compare the activations of attention layers of BERT with WordPiece and Word-level tokenization to assess the effect of including subword tokens. They find that the vocabulary with frequency-based character combinations hinders the ability of modeling semantically meaningful relations between words. Additionally, tokenization based on word occurrence statistics results in representations that are dependent on frequency information rather than semantics [Gong et al. 2018]. On the other hand, it has been proposed to apply subword regularization by utilizing multiple subword segmentation to enhance the robustness of the neural machine translation models [Kudo 2018]. Based on this algorithm, which implements BPE and Unigram language models, SentencePiece has been proposed as another tokenization method [Kudo and Richardson 2018]. Recently, Xu et al. [2021] approached the problem of finding the best token vocabulary with proper size in the scope of the trade-off between vocabulary entropy and vocabulary size. The produced vocabularies in diverse scenarios achieve both reduced sizes and performance improvements. In addition, learning optimal vocabulary takes significantly less time than the regular BPE-search approach.

Alternative tokenization algorithms using morphological analysis can be promising candidates for subword tokenization that increase training efficiency and downstream performance

[Park et al. 2020]. Rule-based tokenization algorithms utilizing lexicons and semantic parsing can extend existing methods to cross-lingual settings [Vasiu and Potolea 2020]. Joint and hybrid tokenization approaches combine coarse and fine-grained representations to incorporate Word-level and subword representations [Hiraoka et al. 2021]. Multi-grained tokenization methods are incorporated into the model architecture to capture multi-word representations, such as *ice cream*, at the expense of increased computational complexity [Zhang et al. 2021a]. Enabling a gradient-based learnable representation in the tokenization step of the pipeline is an emerging line of research [Tay et al. 2021]. In our study, we provide a comprehensive analysis of the impact of tokenization algorithms at different granularity levels from character level to word level, and evaluate the performances on a diverse range of downstream tasks.

2.2 Tokenization in Low-Resource Languages

Tokenization-based methods aiming to enhance the downstream task performance in low-resource languages have been studied before introducing the de facto tokenization methods [Kulick 2011]. With the emergence of the de facto tokenizers, the effects of SentencePiece, Word-level, and Syllable-level tokenization strategies are investigated for low-resource languages, such as Thai [Lowphansirikul et al. 2021]. In addition, Li et al. [2021] show that character-based subword tokenization methods give better results than syllable-based ones in Tibetan-to-Chinese machine translation.

Part-of-Speech (POS) Tagging is one of the downstream tasks in which different tokenization-based methods are employed in low-resource languages [Ding et al. 2019a, 2018; Kaing et al. 2021]. Morphological analysis is used to propose a tokenization system for Kurdish [Ahmadi 2020]. Exploiting pretrained models with parameter freezing and additional intermediate layers is beneficial for Uyghur-Chinese machine translation [Zhang et al. 2021b]. Dossou and Emezue [2021] propose a phrase-based tokenization method for neural machine translation tasks between the Fon language and French. Since the Fon language is quite specific and low resource, bilingual people are involved in data cleaning and preprocessing phases to extract the best phrases based on the linguistic components of the Fon language. Park et al. [2021] study morphological features of the Korean language while training a tokenizer in the scope of machine translation. During training tokenizers, target sentences that are not processed by morphological analysis are also utilized.

In Turkish, morphological segmentation [Üstün and Can 2016] is studied with word embeddings but it does not focus on the Transformer-based language models. We summarize the related studies on low-resource languages in the literature in Table 2 by listing their tokenization methods, languages of interest, and the downstream tasks addressed in the study. Overall, we observe that there is a gap in the literature on tokenization studies focusing on Transformer-based language models, specifically in morphologically rich languages such as Turkish.

2.3 Language Processing for Turkish

Automated methods for Turkish language processing emphasize and exploit its agglutinative and morphologically rich nature. Early studies extend the Morfessor baseline algorithm [Smit et al. 2014] for morphologically rich languages, including Turkish for Speech Recognition [Creutz et al. 2007]. POS tagging gives away clues about the morphological structures of the words, which are also applied for the task of Named Entity Recognition [Yeniterzi 2011]. Several word processing methods, including stemming and truncation, are examined for searching Turkish text collections as well [Can et al. 2008].

More recently, unsupervised morphological segmentation has been used to analyze the effect of representing morphological character combinations with dedicated embeddings (morph2vec) in comparison with character-level embeddings (char2vec) [Üstün et al. 2018]. Character-level

Table 2. A Brief Summary of the Existing Studies Related to Tokenization for Low-Resource Languages

Study	Tokenization Methods	Languages	Downstream Task
Kulick [2011]	Regex, Morphological-level	Arabic	POS Tagging
Üstün and Can [2016]	Morphological-level	Turkish	Segmentation
Ding et al. [2018]	Morpheme-level	Burmese, Khmer	POS Tagging
Ding et al. [2019a]	Syllable-level	Burmese (Myanmar)	POS Tagging
Ahmadi [2020]	Word-level, WordPiece, BPE, Unigram	Kurdish	-
Zhang et al. [2021b]	BPE	Uyghur	Machine Translation
Lowphansirikul et al. [2021]	SentencePiece, Word-level, Syllable-level	Thai	Seq. & Token Classifi.
Li et al. [2021]	BPE, Character-level, Syllable-level	Tibetan	Machine Translation
Kaing et al. [2021]	Character-level	Khmer	POS Tagging
Dossou and Emezue [2021]	Word-level, Phrase-level	Fon	Machine Translation
Park et al. [2021]	Morphological-level, SentencePiece	Korean	Machine Translation

seq2seq modeling is also utilized for Turkish spelling correction [Büyük 2020]. Considering the word ordering in Turkish, Yücesoy and Koç [2019] propose a new weighting scheme for the GloVe [Pennington et al. 2014] word embeddings that increase the performance in analogy tests, especially in the low-resource setup. In terms of contextual embeddings, Tokgoz et al. [2021] evaluate the performance of DistilBERT [Sanh et al. 2019] and BERTurk [Schweter 2020] and find that specialized tokenizers for Turkish provide a performance boost for Text Classification.

Although there have been some efforts to pretrain Turkish language models [Loodos 2020; Schweter 2020] and analyze morphological embeddings [Üstün et al. 2018], the effect of tokenization algorithms, including a Morphological-level one, is yet to be studied for Transformer-based language models.

In order to comparatively assess the effect of different tokenization algorithms with varying vocabulary sizes on Turkish Natural Language Processing tasks, we conduct extensive experiments for both pretraining and fine-tuning of Transformer-based language models.

3 IMPACT OF TOKENIZATION

In order to understand the impact of different tokenization methods on language modeling, we first explain the tokenization approaches that are examined in this study. We then introduce our pipeline that describes the details of various steps to obtain language models with different tokenizers.

3.1 Tokenization Methods

In our study, we consider five tokenization algorithms making use of different linguistic features, including characters, frequency, and grammatical rules, explained with respect to the granularity levels as follows.

- **Character-level:** Unlike the tokenization methods performed on word or subword units, Character-level tokenizers split words into the smallest parts. Since the Character-level tokenizer requires no training to learn a vocabulary, we employ the ByT5 tokenization [Xue et al. 2021]. The advantage of this type of tokenization is that it can be utilized in any language to represent any character sequence at the byte level and enable diverse modeling. Character-level tokenization also reduces the memory requirement in terms of model size, since it has a very limited number of tokens in the vocabulary. A disadvantage of this approach is that the model has to spend more capacity to reach a higher-level representation compared with other tokenization methods. For instance, the language model has to learn during training that “t” and “h” co-occur frequently in English, whereas another tokenizer can provide this information to the models with a “th” token. Furthermore, the output for a given sequence

would contain a large number of tokens when compared to other tokenizers. This results in potential information loss since deep language models have an input parameter of text sequence length.

- **BPE:** Byte Pair Encoding is a frequently used tokenizer for pretrained language models [Sennrich et al. 2016]. The granularity of BPE can be considered as mid-level between character level and word level, such that tokens are mostly subwords depending on vocabulary size. In this method, all unique words are first extracted. A base vocabulary is then constructed from all symbols occurring in the unique words. The final vocabulary is built by merging the symbols according to the frequencies of consecutive symbols or subwords. Since BPE operates with byte representations, the vocabulary can encompass tokens from multiple languages and informal character sequences, such as emojis.
- **WordPiece:** Similar to BPE, WordPiece is also based on merging characters in the documents [Schuster and Nakajima 2012]. Its main difference from BPE is that WordPiece merges symbols towards maximizing a likelihood score of language modeling, that is, when the probability of the merged symbol divided by individual probabilities of the symbols is greater than any other symbol pair. WordPiece and BPE are frequency-based algorithms that aim to increase the modeling power of individual tokens while being able to tokenize words that are not encountered during the training of the tokenizer.
- **Morphological-level:** Morphological analysis can provide suffixes and word stems that are semantically more meaningful and valuable than the tokens obtained with overlapping frequency or likelihood. Therefore, we use the parsing output (without tags) of morphological analysis as input tokens. We use the Zemberek morphological analysis tool for Turkish [Akın and Akın 2007] before training the tokenizer. The advantage of Morphological-level tokenization is to capture grammatically interpretable character sequences in modeling and learn the semantics based on the suffixes of words. A disadvantage of this approach is that word stems are not split further and constitute a large set that has to be included in the vocabulary.
- **Word-level:** The granularity of the Word-level tokenizer is surface forms of words, that is, it splits text according to the spaces between words. Word-level tokenization requires no vocabulary training since one can apply it by just splitting text with white space characters. One explicit disadvantage is that this tokenizer requires more vocabulary size to properly tokenize the same amount of text compared with other methods. Since vocabulary has a limited size in language modeling, out-of-vocabulary or unknown tokens are likely to be observed in this approach.

The sample outputs provided by different tokenization methods are given for a sample sentence "Toplumsal barış sağlanır" (translated as "Social peace would be achieved") in Table 3. All tokenizers have a vocabulary size of 16.6k tokens in this example, except that the Character-level tokenizer has a vocabulary size of 384 characters. We note that BPE and WordPiece tokenizers can assign the surface forms of words to tokens, while the Word-level tokenizer fails to capture some words and produces unknown tokens. The reason could be that the vocabulary capacity is utilized more efficiently by BPE and WordPiece tokenizers, whereas the Word-level tokenizer fills up the vocabulary with more frequent words and cannot tokenize less frequent words. The Morphological-level tokenizer overcomes this by assigning individual tokens to suffixes and isolating the word stems. We note that the output sequence length of Character-level tokenization is considerably higher than other tokenizers, which is not practical when the language model requires a limited length of input text sequence (e.g., if this length parameter is set to 10 tokens, then all methods can properly represent the input, except that the Character-level tokenizer can represent only its first 10 tokens or characters).

Table 3. Outputs of Different Tokenization Methods for a Sample Input, "Toplumsal barış sağlanır" (translated as "Social Peace Would be Achieved")

Method	Tokenized text
Character-level	"t", "o", "p", "l", "u", "m", "s", "a", "l", " ", "b", "a", "r", "ı", "ş", " ", "s", "a", "ğ", "l", "a", "n", "ı", "r"
BPE	"[CLS]", "toplumsal", "barış", "sağ", "##lanır", "[SEP]"
WordPiece	"[CLS]", "toplumsal", "barış", "sağlan", "##ır", "[SEP]"
Morphological-level	"[CLS]", "toplum", "##sal", "barış", "sağ", "##lanır", "[SEP]"
Word-level	"[CLS]", "[UNK]", "barış", "[UNK]", "[SEP]"

3.2 Our Pretrained Model: RoBERTa-Turkish-medium

We develop a pipeline, illustrated in Figure 1, which consists of collecting and cleaning the training corpus, training a tokenizer with a fixed-length vocabulary, and pretraining a deep language model by using the selected tokenizer and its vocabulary. We are then able to fine-tune the model on different downstream tasks to evaluate the performance of the tokenizer. The details of our pipeline in Figure 1 are described as follows.

- **Obtain the corpus.** We use the OSCAR deduplicated corpus for pretraining our language model [Huggingface 2021; Ortiz Suárez et al. 2019]. OSCAR is a multilingual corpus that is obtained by filtering the Common Crawl corpus, which maintains an open repository of publicly available web pages. We use the split of this corpus prepared for Turkish.
- **Preprocess the corpus.** We observe that the Turkish split of OSCAR includes many documents in languages other than Turkish. We thereby filter out 95,152 documents that are not in Turkish by using an automated language detector [Shuyo 2010]. The filtering process results in 11,501,370 documents for pretraining.
- **Train the tokenizer and obtain a vocabulary.** The tokenization process, depicted inside a dashed rectangle in the figure, is conducted in three steps: (i) Applying normalization to clear the invalid characters from the text, (ii) training the tokenizer according to a predetermined vocabulary size (except for the Character-level tokenizer since it depends on a set of characters rather than a vocabulary), and (iii) processing the corpus with the trained tokenizer to obtain tokenized pretraining data. We apply lowercase conversion and NFC normalization.[2]
- **Pretrain the model.** Next, we pretrain the language model on the corpus with the trained tokenizer. This step is computationally more expensive than the following fine-tuning step, which helps specialize the model for certain downstream tasks.

The architecture of the language model in the following step is similar to BERT [Devlin et al. 2019] but smaller in size. The underlying structure draws from Transformer encoders [Vaswani et al. 2017] and operates bidirectionally. The model employs the self-attention algorithm that calculates an attention function between the vector representations of tokens. Calculation of the attention scores, given in Equation (1), involves query (Q), key (K), and value (V) matrices with tunable parameters that are learned in a self-supervised fashion:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \tag{1}$$

The d_k term is the dimension of the key matrix. It provides a scaling to the attention mechanism, known as the scaled dot-product attention. The attention scores are utilized in conjunction with the tokenized input in an embedding layer (EmbLayer), then passed to a layer normalization (LayerNorm) and feedforward network (FeedForw), then, finally, to an output layer (Output), as

[2]Unicode normalization is important for Turkish since there are special characters (ç, ğ, ı, ö, ş, ü) in the Turkish alphabet that are not observed in English. We note that NFC Unicode normalization provides all letters in Turkish.

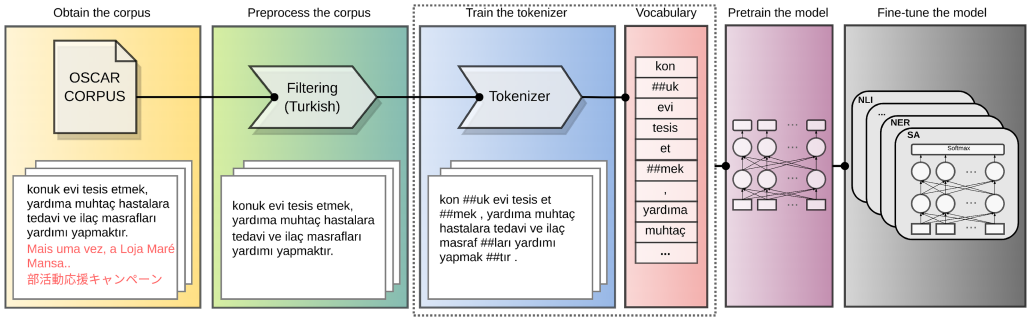


Fig. 1. Illustration of our pretraining pipeline. The translation for the example sentence in the figure is "establishing a guest house is to pay the medication costs for patients in need of treatment." There are five steps in the pretraining process. We first obtain a text corpus (*OSCAR Turkish deduplicated*) and preprocess the corpus by filtering non-Turkish texts. We then choose a tokenization algorithm and implement it on the filtered corpus. We obtain a vocabulary from the trained tokenizer. We are then able to pretrain a deep language model (*RoBERTa-TR-medium*) using the pretrained tokenizer and obtained vocabulary. Last, we fine-tune our model on several downstream tasks, including Sentiment Analysis (SA) and Named Entity Recognition (NER).

given in Equation (2):

$$\begin{aligned} X &= EmbLayer(Tokenized\ Input), \\ X' &= LayerNorm(X + Attention(X)), \\ X'' &= LayerNorm(X' + FeedForw(X')), \\ Y &= Output(X''). \end{aligned} \tag{2}$$

This architecture is stacked multiple times, known as layers, and the overall structure is duplicated multiple times with different weight initializations, known as attention heads. The number of attention heads and the number of layers in the network are hyperparameters that determine the size of the model and the computational complexity of the pretraining and fine-tuning procedures.

We pretrain language models using Turkish (TR) text, with the RoBERTa pretraining procedure and configuration [Liu et al. 2019], but smaller in terms of the number of layers, attention heads, and hidden size (we follow the same architecture as BERT-medium [Devlin et al. 2019]). We thereby call the model *RoBERTa-TR-medium*. We determine the vocabulary size based on the number of parameters of the models. Similar to BERT [Devlin et al. 2019], the number of parameters associated with the vocabulary constitutes 20% of the whole model. The vocabulary size for tokenizers is therefore 16.6k tokens, except for Character-level. The calculation of vocabulary size is given as $|V| = (M \times R)/H$, where $|V|$ is the number of tokens in the vocabulary (i.e., the vocabulary size), M is the number of total parameters in the language model, R is the ratio of the vocabulary size to the whole model, and H is the hidden dimension size (in our case, M is approximately 42.7 and H is 512).

The pretraining details of our medium model are given in Table 4. Since we examine the effect of different tokenization strategies in Turkish, we keep the pretraining procedure computationally simpler because extensive pretraining might overshadow possible advantages of tokenization algorithms. When a model is extensively pretrained, the performance can converge to high scores, even with Character-level encoding [Xue et al. 2021]. Nevertheless, we compare the results of our model with the current state-of-the-art performance for a sanity check, that is, the rationality of our results. To do so, we employ the BERTurk model [Schweter 2020], which is a Turkish pretrained version of BERT-base [Devlin et al. 2019]. Therefore, we provide the configuration of

Table 4. Details of Pretraining Configurations for BERTurk and Our Model, *RoBERTa-TR-medium*

	BERTurk-base	RoBERTa-TR-medium
Parameters	110.62 M	42.69 M
Train data	35 GB	27 GB
Layers	12	8
Heads	12	8
Hidden size	768	512
Batch size	n/a	264
Max length	512 tokens	514 tokens
Train time	9.63 days	2 days*
Hardware	TPU v3-8	2x Nvidia RTX2080 Ti

(*Train time and hardware are given for a vocabulary size of 16.6k tokens. Train time can differ for other vocabulary sizes as we report detailed information in Section 5).

BERTurk along with our model's configuration in the table (we did not pretrain BERTurk, but fine-tuned it on our downstream tasks).

We use the AdamW [Loshchilov and Hutter 2019] optimizer (β_1 is 0.90, β_2 is 0.98, and ϵ is 1e-6), linear scheduling with a warmup ratio of 1e-2 and peak learning rate of 5e-5, and gradient accumulation with 22 steps. Other hyperparameters are set to the RoBERTa configuration [Liu et al. 2019].

3.3 Fine-tuning Tasks

The performances of the pretrained models with different tokenizers are evaluated by fine-tuning the models on six downstream tasks. The tasks and datasets used for fine-tuning are explained as follows.

- **News Classification:** Given a set of news articles, this task aims to classify each document into a predetermined set of classes, that is, the task is text sequence classification. We use Turkish news classification datasets provided by Toraman et al. [2011]. Merging two datasets from two different news resources results in approximately 7.5k news instances. The news articles are given under eight news topics or categories: sports, economy, national, world, politics, columnists, health, and culture-art.
- **Hate Speech Detection:** The aim of this task is to determine whether a given text sequence includes hate speech towards other individuals or communities with different backgrounds. Hate Speech Detection is a challenging problem with a limited number of resources in the literature since there is no decisive consensus on the definitions of hate or offensive speech and hate language can have various forms in natural language. In this study, we use a recent hate speech dataset in Turkish curated by Toraman et al. [2022]. Data instances are tweets from different hate speech domains, including gender, religion, and politics. There are 100k tweets distributed equally among five hate speech domains annotated as hate, offensive, and normal.
- **Sentiment Analysis:** Sentiment Analysis is a task of text sequence classification to find the author's sentimental state. We use a Turkish dataset, including movie reviews prepared by Demirtas and Pechenizkiy [2013]. The reviews are labeled as having positive and negative sentiments. The dataset is balanced, containing approximately 5.3k instances for each sentiment class.

- **Named Entity Recognition:** Named Entity Recognition is a token classification task to predict predetermined named entities in a text sequence, such as person and location. We use the benchmark dataset [Tür et al. 2003], including Turkish news articles. The dataset contains approximately 32.5k sentences and three named entity classes given as person, location, and organization. The named entities in the dataset are annotated with the IOB2 [Ramshaw and Marcus 1995] tags, such that each entity chunk starts with B-<class>, and continues with I-<class>, for example, New York has the tags of B-<LOCATION> and I-<LOCATION>.
- **Semantic Text Similarity:** Semantic similarity between two text sequences is measured in this task. The sentence pairs are annotated on a scale between 0 (i.e., no semantic similarity) and 5 (i.e., semantically equivalent) according to their similarity degree. In contrast to the classification tasks, STS is handled as a regression problem. To evaluate the performance of the model, the correlation between the ground truth and model predictions is taken into consideration. We use a Turkish STS dataset that is the translation of the STSb dataset [Beken Fikri et al. 2021]. The domain of the sentence pairs varies from news articles to online forum messages. The dataset includes approximately 8.6k sentence pairs in total.
- **Natural Language Inference:** Given two sentences, the aim of Natural Language Inference is to predict whether the latter is inferred by the former. The dataset includes three types of semantic relations: The first sentence can entail the second one (*entailment*), the sentences can be irrelevant to each other (*neutral*), or the first sentence can contradict the second one (*contradiction*). We use a Turkish NLI dataset, which is the translated version of the SNLI dataset [Budur et al. 2020]. The dataset includes approximately 570k sentence pairs.

4 EXPERIMENTS AND DISCUSSIONS

We conduct two experiments in this study. First, we compare the performances of tokenization methods for Turkish downstream tasks. Second, we analyze the effect of vocabulary size on the downstream task performance. For each experiment, we describe our experimental design and report the results.

4.1 Comparison of Tokenization Methods

4.1.1 Experimental Design. We compare the performance of tokenizers using *RoBERTa-TR-medium* on Turkish downstream tasks (**RQ-1**). We use a fixed size of vocabulary, 16.6k tokens, in this experiment to understand how different tokenizers would perform under the same configuration. We analyze the performance of our medium-size model in comparison with a larger state-of-the-art model. To do so, we report the performance of BERTurk [Schweter 2020], a Turkish model with a size similar to BERT-base [Devlin et al. 2019].

We fine-tune the models, which are pretrained using different tokenizers, on the downstream tasks given in Section 3.3. For fine-tuning our models, the configurations and hyperparameters along with dataset sizes are given in Table 5. We measure weighted precision, recall, and F1 score for all tasks except STS, in which the Pearson correlation is reported with the p-value. We apply 10-fold cross-validation and report the average scores. We determine statistically significant differences between the means, which follow non-normal distributions, using the two-sided Mann-Whitney U (MWU) test at a 95% interval with Bonferroni correction.

4.1.2 Experimental Results. We report the fine-tuning results in Table 6. Our main observations and answers to **RQ-1** can be summarized as follows.

WordPiece and BPE are the highest performing tokenizers in Turkish language modeling. WordPiece and BPE are de facto standard tokenizers that are practically employed in language modeling. Indeed, we find that WordPiece statistically significantly outperforms other tokenizers

Table 5. Details of Fine-tuning Configurations for Different Downstream Tasks

		News Classification	Hate Speech Detection	Sentiment Analysis	Named Entity Recognition	Semantic Text Similarity	Natural Language Inference
R-TR-m	Epochs	10	5	10	10	10	3
	Max. length	514	256	514	514	514	514
	Batch size	32	32	32	16	16	16
BERT	Epochs	3	3	3	3	25	3
	Max. length	256	256	256	256	256	256
	Batch size	32	32	32	16	32	16
	Learning rate	1e-5	1e-5	1e-5	1e-5	1e-5	1e-5
	Train Size	6,786	90,000	9,595	29,332	7,765	512,130
	Test Size	754	10,000	1,067	3,259	863	56,903

R-TR-m refers to our model, RoBERTa-Turkish-medium, and *BERT* refers to BERTurk. The task configurations can change due to space complexity and data size. We apply a constant learning rate for all tasks except the linear decay learning rate in NLI. When Character-level tokenizer is used, we set the number of epochs to 10, max. sequence length to 1024, and batch size to 12, for all tasks.

Table 6. Fine-tuning Results of Different Tokenizers (rows) on Six Downstream Tasks (Columns) using Turkish Datasets

		News Classification			Hate Speech Detection			Sentiment Analysis			Named Entity Recognition			Semantic Text Similarity		Natural Language Inference		
		P	R	F1	P	R	F1	P	R	F1	P	R	F1	corr	p-value	P	R	F1
	BERT	0.918	0.917	0.917	0.781	0.781	0.781	0.927	0.927	0.927	0.935	0.955	0.945	0.862	<1e-178	0.852	0.852	0.852
R-TR-medium	Char	0.715	0.723	0.713	0.606	0.609	0.607	0.812	0.812	0.812	0.730	0.788	0.757	0.256	<1e-4	0.620	0.619	0.619
	BPE	**0.886**	**0.885**	**0.885** •	0.742	0.737	0.738	0.882	0.881	0.881 ○	0.851	0.883	0.866 ○	0.487	<2e-32	0.772	0.772	0.772
	WP	0.882	0.881	0.881 ○	**0.745**	**0.745**	**0.745** •	**0.884**	**0.884**	**0.884** •	**0.858**	**0.893**	**0.875** •	**0.718**	<3e-92 •	**0.778**	**0.778**	**0.778** •
	Morph	0.869	0.868	0.867	0.726	0.727	0.726	0.824	0.823	0.823	0.839	0.872	0.855	0.655	<5e-63 ○	0.768	0.768	0.768
	Word	0.857	0.857	0.856	0.647	0.649	0.648	0.805	0.805	0.805	0.791	0.740	0.764	0.492	<2e-16	0.603	0.598	0.595

The average of 10-fold cross-validation is reported in terms of weighted precision (P), recall (R), and F1 score. *BERT* refers to BERTurk, which is structurally similar to BERT-base, but pretrained for Turkish text. For STS, Pearson correlation (corr) is reported with the p-value. *R-TR-medium* refers to our pretrained model for Turkish text, *RoBERTa-Turkish-medium*, along with each tokenization method. *Char* refers to Character-level tokenizer, *BPE* refers to Byte Pair Encoding, *WP* refers to WordPiece, *Morph* refers to Morphological-level tokenizer, and *Word* refers to Word-level tokenizer. The highest score among tokenizers for each task is given in bold. The symbol "•" indicates statistically significant difference at a 95% interval (with Bonferroni correction $p < 0.0125$) in pairwise comparisons between the highest performing method and others (except the ones with "○").

in most of the tasks in our experiments for Turkish. The only exceptions are that BPE has higher scores in News Classification, but the difference between the performances of BPE and WordPiece is not statistically significant in that case. Similarly, WordPiece has a higher score than BPE in Sentiment Analysis and Named Entity Recognition, but the differences are not statistically significant as well. We thereby argue that WordPiece and BPE are the highest-performing tokenizers in Turkish, noting that WordPiece has better performance than BPE in the majority of tasks. The results emphasize the proper choice of the tokenizer algorithm in terms of the downstream task performance.

The Word-level tokenizer performs poorly due to many unknown tokens. The Word-level tokenizer performs poorly compared with the BPE, WordPiece, and Morphological-level tokenizers, possibly due to poor utilization of the available vocabulary capacity. Therefore, we examine the ratio of the unknown tokens to all tokens in the fine-tuning datasets and report them in Table 7. It is apparent that almost half the tokens are unknown to the model when the Word-level tokenizer is used in all tasks. The reason why Word-level still achieves comparable results with other tokenizers might be that the model is trained with the Masked Language Modeling task and gains an ability to infer meaning even with many unknown tokens. We argue that increasing the vocabulary size can offer a trade-off that reduces the number of unknown tokens at the cost of applying more parameter updates per token embedding.

Table 7. Ratios of Unknown Tokens to All Tokens in the Fine-tuning Datasets in Table 6

	News Classification	Hate Speech Detection	Sentiment Analysis	Named Entity Recognition	Semantic Text Similarity	Natural Language Inference
BPE	0.000	0.000	0.000	0.000	0.000	0.000
WordPiece	1.447e-6	4.292e-6	4.824e-6	0.000	0.000	0.000
Morph-level	0.021	0.171	0.183	0.018	0.024	0.008
Word-level	0.587	0.515	0.502	0.457	0.522	0.457

Table 8. Tokenization Output of True Parsing and Morphological Analysis tool, Zemberek, Considering Turkish Syntax Rules

	Output			
Sample Sentence	İstanbullular	güneşin	tadını	çıkarabildiler
True Parsing	İstanbul ##lu ##lar	güneş ##in	tat ##ı ##nı	çık ##ar ##abil ##di ##ler
Zemberek	İstanbullular	güneş ##in	tad ##ın ##ı	çıkar ##abil ##di ##ler

Sample sentence is "İstanbullular güneşin tadını çıkarabildiler" (translated as "People of Istanbul were able to enjoy the sun").

Morphological-level has competitive results with state-of-the-art tokenizers. The differences between the weighted F1 scores of the Morphological-level tokenizer and the best-performing tokenizer are statistically significant but very small (between 0.01 and 0.02) in all tasks, except that it is approximately 0.06 in Sentiment Analysis and Semantic Text Similarity (STS). However, this difference is not statistically significant in STS. We thereby argue that the Morphological-level tokenizer achieves competitive results with de facto tokenizers. Moreover, the performance of the Morphological-level tokenizer is always better than those of Character-level and Word-level tokenizers. We argue that suffixes can provide useful information for language modeling in Turkish. The poor performance compared with de facto tokenizers can be attributed to two observations. First, the method has a dependency on the performance of the morphological analyzer, Zemberek [Akın and Akın 2007], which we employ for obtaining prefixes and suffixes. We observe possible errors such as wrong morphemes in the output of the morphological analyzer, as reported in Table 8. For instance, the word "İstanbullular" (translated as "People of Istanbul") is not tokenized correctly; however, the word contains inherited information, such as hometown (##lu) and plural (##lar). Second, the Morphological-level tokenizer is limited in terms of word stems since roots are not split into smaller pieces in this approach, increasing the likelihood of observing unknown tokens, as observed in Table 7.

Character-level tokenizer has no significant benefit. The Character-level tokenizer achieves the worst performance for Turkish in most tasks. The reason could be that our medium models might be inadequate to comprehend the relations among characters, which could be better modeled by larger language models [Xue et al. 2021]. However, we argue that the size and architecture of larger models could be inefficient to outperform de facto tokenizers, as we address in Section 3.1.

Medium models can be competitive with larger ones. We expect that the performance of our medium models is lower than larger models, that is, BERTurk, due to the computational advantages of larger models. However, we find that the performance gap is narrow for particular tasks. Our 3-times-smaller model recovers 97% of BERTurk's performance in News Classification, 95% in Hate Speech, 95% in Sentiment Analysis, 93% in Named Entity Recognition, 83% in Semantic Textual Similarity, and 91% in Natural Language Inference. One possible reason for the relatively lower recovery score of the STS task is that it is a regression task. In classification, output logits are mapped to classes. Since there is no such quantization in regression, there can be more deviations in the correlation between ground truths and predictions. Therefore, we argue that our medium-sized

models can be suitable in downstream applications with a limited computation budget, similar to model distillation [Sanh et al. 2019].

Hate Speech Detection, Semantic Text Similarity, and Natural Language Inference are challenging tasks. The most successful models, excluding the BERT-base model, achieve lower than 0.8 with respect to their performance metric in Hate Speech Detection, Semantic Text Similarity, and Natural Language Inference. We argue that these tasks are not likely to be performed with simple keyword matching, but they require advanced models having a broader understanding of language. We note that WordPiece tokenization outperforms other tokenizers in all of these tasks.

4.2 Analysis of Vocabulary Size

4.2.1 Experimental Design. In the previous experiments, we fixed the vocabulary size for all tokenizers except the Character-level tokenizers. However, vocabulary embeddings contribute to the total number of model parameters and the effect of the vocabulary size can vary among different tokenizers (**RQ-2**). We thereby designed an experiment that measures the performances of tokenizers with changing vocabulary sizes. We note that in the ultimate case, when the vocabulary size tends to be infinite, every possible character combination in the corpus is assigned a representation in the vocabulary. In such a case, the modeling becomes similar to conventional word embeddings, such as word2vec [Mikolov et al. 2013]. On the other extreme, the need for contextual representation of a given token increases as vocabulary size gets smaller. In other words, a single token is expected to reflect a wide variety of contextual meanings due to limited vocabulary size.

In this experiment, we fix the hyperparameters of the Transformer blocks in the architecture, for example, the number of layers and hidden size, and adjust the vocabulary size such that the number of parameters attributed to the vocabulary constitutes 10%, 20%, 30%, 40%, and 50% of the entire model. Since the Character-level tokenizer has a fixed vocabulary size, we exclude it from this experiment.

This analysis requires training a separate language model for five vocabulary sizes and four tokenization methods, resulting in a total number of 20 models. Considering six downstream tasks, we would have 120 experimental runs. Therefore, we decided to select two important tasks among six tasks for the sake of efficiency and smaller carbon footprint. We selected a text classification task, Sentiment Analysis, and a token classification task, Named Entity Recognition. In fine-tuning, we apply 10-fold cross-validation and report the average of weighted F1 scores.

4.2.2 Experimental Results. The results of varying vocabulary sizes for different tokenizers are given in Figure 2. We report the vocabulary sizes by removing the fractional parts of decimal numbers (e.g., 16.6k is given as 16k). Our main observations and answers to **RQ-2** are listed as follows.

We observe an increasing pattern for the performance of all tokenizers as the vocabulary size increases. Increasing vocabulary size can result in less number of unknown tokens for Morphological-level and Word-level tokenizers. BPE and WordPiece can benefit from higher level tokens by merging subword tokens when vocabulary size increases, for example, *New Y ##or ##k* can be represented as *New York* when vocabulary size is sufficient. In terms of downstream tasks, Named Entity Recognition can benefit from increasing vocabulary size more than Sentiment Analysis since Named Entity Recognition is a token classification task that depends on the performance of predicting individual tokens. The reason for smaller improvements in Sentiment Analysis could be that it is a sequence classification task that can tolerate individual unknown tokens while predicting the sentiment of the given sequence.

Performance improvement gets faster saturation for de facto tokenizers compared with others. De facto tokenizers, that is, BPE and WordPiece, do not dramatically benefit from

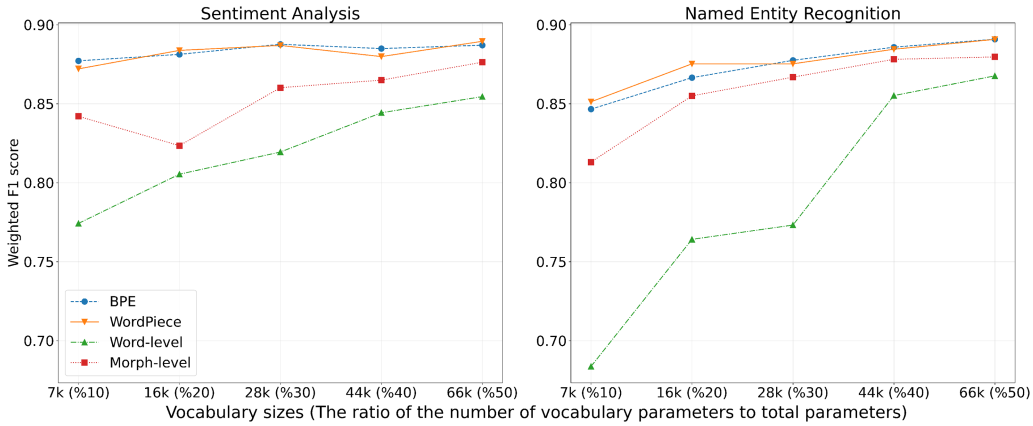


Fig. 2. The effect of varying the vocabulary size for different tokenization methods. The results are reported for a text classification task, Sentiment Analysis, and a token classification task, Named Entity Recognition.

larger vocabulary sizes, which indicates a saturation in performance at smaller sizes. A potential cause for this saturation might be that BPE and WordPiece can tokenize almost any input in the fine-tuning datasets with a vocabulary size greater than 16k (i.e., the ratios of unknown tokens to all tokens are mostly zeros in Table 7). Furthermore, they outperform Morphological-level and Word-level tokenizers in both tasks. However, the performance gap diminishes as the vocabulary size increases, probably due to fewer unknown tokens. This also reduces the impact of tokenization on the model performance. In terms of downstream tasks, while Sentiment Analysis has performance saturation in the small number of vocabulary sizes (e.g., 7k and 16k), Named Entity Recognition gets saturation after 16k, probably due to the fact that it is a token classification task.

Very large vocabulary sizes have a less practical advantage, specifically for de facto tokenizers. When the vocabulary size exceeds 66k tokens, performance improvement may become infeasible due to increasing computational complexity. Instead, computational resources can be dedicated to increasing other resources, such as the number of parameters in the Transformer blocks. For de facto tokenizers, this situation is more apparent because their performances are already saturated at even smaller vocabulary sizes. We observe that the ratio of the number of vocabulary parameters to the total number of model parameters can be empirically chosen as 20% for de facto tokenizers and 40% for others in order to satisfy the trade-off between model size and performance.

Subword frequencies are more effective than morphological features for small vocabularies. The BPE and WordPiece tokenizers outperform the Morphological-level tokenizer, specifically in smaller vocabulary sizes. While de facto tokenizers incorporate the highest frequently observed subwords, the Morphological-level tokenizer accommodates a wide variety of suffixes. Therefore, the Morphological-level tokenizer might not be able to include some of the important subwords in small vocabularies. However, the differences between the performances of these tokenizers are getting smaller as the dictionary size increases.

5 BROADER IMPACT AND ETHICAL CONCERNS

Since we pretrain large language models and fine-tune them on several downstream tasks for experimental evaluations, we would like to emphasize the broader impact and ethical concerns [Baeza-Yates 2022; Bender et al. 2021; Mitchell et al. 2019]. We thereby provide a discussion on our study's broader impact, transparency, responsibility, and fairness in this section.

5.1 Broader Impact

We anticipate that our study will have a broader impact on the research community that focuses on the impact of tokenization and vocabulary size on language modeling. Although the models are medium-sized, we observe that their downstream performances are comparable with base models (see Table 6). Our research elaborates on an agglutinative low-resource language, Turkish, and our findings can provide guidance to other researchers who study similar languages. We also provide performance measurements for a diverse set of downstream tasks, which can be useful for various real-life applications, such as recommendation systems and finance.

5.2 Transparency

In order to provide a transparent modeling [Mitchell et al. 2019], we explain all details regarding text corpus used in pretraining our language models in Section 3.2 as well as the details of the algorithms used throughout obtaining the language models in Section 3.1 and model configurations in Section 3.2. The details of how we conduct the experimental evaluations are also reported in Sections 3.3 and 4.

5.3 Responsibility

There is an increasing environmental awareness among the Machine Learning community about responsible training, such as the carbon footprint of extensive model training [Bender et al. 2021; Henderson et al. 2020]. We estimate the carbon footprint of our study based on the energy usage of GPUs for pretraining and fine-tuning, reported in Tables 9 and 10, respectively. We report execution time in hours and electrical energy consumption in kWh. It is assumed that the power consumption during training is equal to the maximum power drain of GPUs and that they operate at maximum power utilization (250W). This estimation ignores the carbon footprint of CPU utilization and the manufacturing costs of the hardware. We note that the emissions for different tokenization methods are close to each other and report the cost for a single tokenization method, WordPiece, in this analysis.

Based on the calculations of execution time and energy consumption, we estimate the carbon footprint of our models in terms of greenhouse gas (GHG) emission in kg CO_2eq. We then plot the carbon footprint of pretraining with different vocabulary sizes and fine-tuning on different downstream tasks in Figure 3. We note that for the GHG emission of pretraining, the vocabulary sizes given at the x-axis have an exponential scale, starting from 7k to 66k. We observe a linearly increasing trend in GHG emissions as the vocabulary size increases, which indicates an exponential growth in environmental damage. We underline that the performance gain in Figure 2 comes at the cost of increasing environmental damage; therefore, we suggest that a reasonably smaller vocabulary size is a preferable choice for pretraining. We further observe that the greenhouse gas emissions caused by the fine-tuning experiments are in correlation with the size of the utilized training set.

For the conversion of electrical energy usage to CO_2eq GHG emission, we use a local conversion factor specified by the Turkish government [Republic of Turkey 2020]. The specified value is an upper bound of the spontaneous value reported by Electricity Maps [ElectricityMap 2022]. Based on our estimation for the GHG release, a lower bound for the social carbon cost (SCC) of the experiments in our study can be approximated as \$117.42 (391.4 kg CO_2eq) with a value of \$300 per ton of CO_2 [Kikstra et al. 2021].

5.4 Fairness

The pretraining text corpus in our study, the OSCAR corpus, has millions of web page texts, a part of which may include biased content ignoring the fairness principle towards all communities and

Table 9. Energy Consumption for Pretraining with a Single Tokenization Method for Different Vocabulary Sizes

	Vocabulary Size				
	7k	**16k**	**28k**	**44k**	**66k**
GPU Hours (h)	2 × 36.3h	2 × 40h	2 × 44h	2 × 52.5h	2 × 57.75h
Energy Consumption (kWh)	18.15	20.00	22.00	26.25	28.88

Table 10. Energy Consumption for Fine-tuning with a Single Tokenization Method for Different Downstream Tasks

	News Classification	**Hate Speech Detection**	**Sentiment Analysis**	**Named Entity Recognition**	**Semantic Text Similarity**	**Natural Language Inference**
GPU Hours (h)	2 × 1.77h	2 × 6.08h	2 × 2.50h	1 × 8.33h	2 × 2.05h	2 × 35.00h
Energy Consumption (kWh)	0.89	3.04	1.25	2.08	1.03	17.50

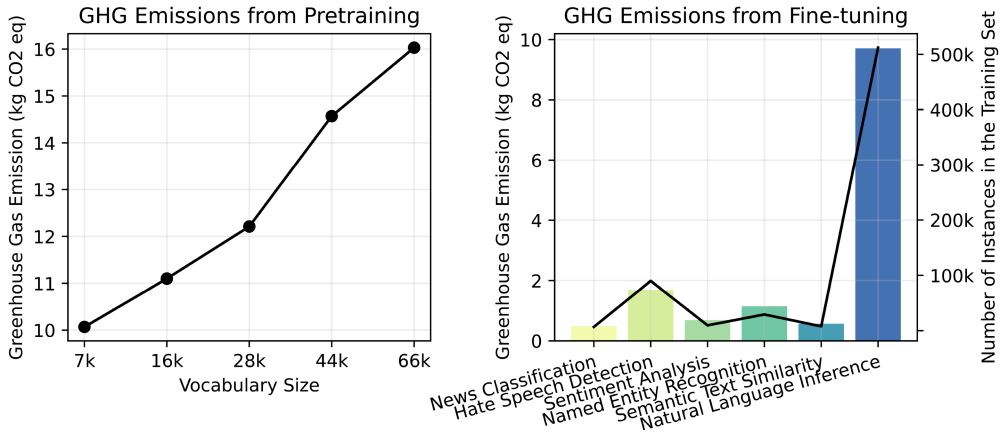


Fig. 3. Carbon footprint (in terms of kg CO_2) of pretraining with a single tokenization method for different vocabulary sizes (left) and fine-tuning for different downstream tasks (right). The numbers of training instances used in fine-tuning are represented by a line at the right subplot with their corresponding values at the second y-axis.

individuals with a variety of backgrounds and profiles. Moreover, the fairness of the tokenization algorithms and Transformer-based language models is still in debate [Baeza-Yates 2022]. We are not able to make a judgment on the fairness of the corpus and model since, to the best of our knowledge, there is no available automatic tool to assess fairness. Nevertheless, we acknowledge that researchers and practitioners should provide fair algorithms, data, and models. The bias can be removed by filtering the pretraining corpus or analyzing the fairness of algorithms used in this study. However, we leave the study of such filtering and analysis to future work since it would require a dedicated effort to develop such algorithms and the scope of this study is to compare empirical performance of different tokenizers and vocabulary sizes.

6 CONCLUSION

We provide a comprehensive study that examines the impact of tokenization in Turkish, which is a low-resource language with a limited number of pretrained deep language models. In order to accomplish this task, we pretrain a medium-sized language model, called *RoBERTa-TR-medium*, with different tokenization algorithms and varying vocabulary sizes. Our language models are

publicly available so that other researchers and practitioners can benefit from our models. This would provide less electrical energy and memory usage with a better carbon footprint in return.

Our experimental results, supported by statistical tests, can shed light on the role of tokenization in language modeling, specifically for morphologically rich languages. We find that the Morphological-level tokenizer is competitive with the de facto tokenizers BPE and WordPiece. Our models, which are approximately three times smaller than state-of-the-art larger models, can recover 97% of the performance of the larger ones. We also show that increasing the vocabulary size improves the performance of Morphological- and Word-level tokenizers more than that of de facto tokenizers. We suggest that the ratio of the number of vocabulary parameters to the total number of model parameters can be empirically chosen as 20% for de facto tokenizers and 40% for others for the trade-off between model size and performance.

In future work, we plan to extend our experiments to other agglutinative languages, such as Finnish and Hungarian, and other tokenization algorithms, such as SentencePiece [Kudo 2018]. Morphological disambiguation [Hakkani-Tür et al. 2018] can be used to improve the quality of morphological analysis, yielding potential improvements in Morphological-level tokenization. We also plan to focus more on artificial intelligence ethics regarding the impact of tokenization on pretraining language models, including, but not limited to, filtering bias in pretraining text corpora and analysis of tokenization algorithms in terms of fairness.

REFERENCES

Sina Ahmadi. 2020. A tokenization system for the Kurdish language. In *Proceedings of the 7th Workshop on NLP for Similar Languages, Varieties and Dialects*. International Committee on Computational Linguistics (ICCL), Barcelona, Spain (Online), 114–127.

Ahmet Afsin Akın and Mehmet Dündar Akın. 2007. Zemberek, an open source NLP framework for Turkic languages. *Structure* 10 (2007), 1–5.

Ricardo Baeza-Yates. 2022. Ethical challenges in AI. In *Proceedings of the 15th ACM International Conference on Web Search and Data Mining (WSDM) (Virtual Event, AZ, USA)*. Association for Computing Machinery, 1–2. https://doi.org/10.1145/3488560.3498370

Figen Beken Fikri, Kemal Oflazer, and Berrin Yanikoglu. 2021. Semantic similarity based evaluation for abstractive news summarization. In *Proceedings of the 1st Workshop on Natural Language Generation, Evaluation, and Metrics (GEM'21)*. Association for Computational Linguistics, Online, 24–33. https://doi.org/10.18653/v1/2021.gem-1.3

Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the dangers of stochastic parrots: Can language models be too big?. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency* (Virtual Event, Canada) *(FAccT'21)*, Association for Computing Machinery, 610–623. https://doi.org/10.1145/3442188.3445922

Kaj Bostrom and Greg Durrett. 2020. Byte pair encoding is suboptimal for language model pretraining. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, Online, 4617–4624. https://doi.org/10.18653/v1/2020.findings-emnlp.414

Emrah Budur, Rıza Özçelik, and Tunga Güngör. 2020. Data and representation for Turkish natural language inference. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP'20)*. Association for Computational Linguistics, Online, 8253–8267. https://doi.org/10.18653/v1/2020.emnlp-main.662

Osman Büyük. 2020. Context-dependent sequence-to-sequence Turkish spelling correction. *ACM Transactions on Asian and Low-Resource Language Information Processing* 19, 4 (2020), 1–16.

Fazli Can, Seyit Kocberber, Erman Balcik, Cihan Kaynak, H. Cagdas Ocalan, and Onur M. Vursavas. 2008. Information retrieval on Turkish texts. *Journal of the American Society for Information Science and Technology* 59, 3 (2008), 407–421.

Mathias Creutz, Teemu Hirsimäki, Mikko Kurimo, Antti Puurula, Janne Pylkkönen, Vesa Siivola, Matti Varjokallio, Ebru Arisoy, Murat Saraçlar, and Andreas Stolcke. 2007. Morph-based speech recognition and modeling of out-of-vocabulary words across languages. *ACM Transactions on Speech and Language Processing* 5, 1 (2007), 1–29.

Erkin Demirtas and Mykola Pechenizkiy. 2013. Cross-lingual polarity detection with machine translation. In *Proceedings of the 2nd International Workshop on Issues of Sentiment Discovery and Opinion Mining* (Chicago, Illinois) *(WISDOM'13)*. Association for Computing Machinery. https://doi.org/10.1145/2502069.2502078

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the*

Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). Association for Computational Linguistics, Minnesota, 4171–4186. https://doi.org/10.18653/v1/N19-1423

Chenchen Ding, Hnin Thu Zar Aye, Win Pa Pa, Khin Thandar Nwet, Khin Mar Soe, Masao Utiyama, and Eiichiro Sumita. 2019a. Towards Burmese (Myanmar) morphological analysis: Syllable-based tokenization and part-of-speech tagging. *ACM Transactions on Asian and Low-Resource Language Information Processing* 19, 1 (2019). https://doi.org/10.1145/3325885

Chenchen Ding, Masao Utiyama, and Eiichiro Sumita. 2018. NOVA: A feasible and flexible annotation system for joint tokenization and part-of-speech tagging. *ACM Transactions on Asian and Low-Resource Language Information Processing* 18, 2 (2018). https://doi.org/10.1145/3276773

Shuoyang Ding, Adithya Renduchintala, and Kevin Duh. 2019b. A call for prudent choice of subword merge operations in neural machine translation. In *Proceedings of Machine Translation Summit XVII: Research Track*. European Association for Machine Translation, Dublin, Ireland, 204–213.

Bonaventure F. P. Dossou and Chris C. Emezue. 2021. Crowdsourced phrase-based tokenization for low-resourced neural machine translation: The case of Fon language. *arXiv preprint arXiv:2103.08052* (2021).

ElectricityMap. 2022. Climate Impact by Area. Retrieved March 16, 2022 from https://app.electricitymap.org/map.

Chengyue Gong, Di He, Xu Tan, Tao Qin, Liwei Wang, and Tie-Yan Liu. 2018. FRAGE: Frequency-agnostic word representation. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems (Montreal, Canada) (NIPS'18)*. Curran Associates Inc., Red Hook, NY, 1341–1352.

Dilek Zeynep Hakkani-Tür, Murat Saraçlar, Gökhan Tür, Kemal Oflazer, and Deniz Yuret. 2018. *Morphological Disambiguation for Turkish*. Springer International Publishing, Cham, 53–67. https://doi.org/10.1007/978-3-319-90165-7_3

Peter Henderson, Jieru Hu, Joshua Romoff, Emma Brunskill, Dan Jurafsky, and Joelle Pineau. 2020. Towards the systematic reporting of the energy and carbon footprints of machine learning. *Journal of Machine Learning Research* 21, 248 (2020), 1–43.

Tatsuya Hiraoka, Sho Takase, Kei Uchiumi, Atsushi Keyaki, and Naoaki Okazaki. 2021. Joint optimization of tokenization and downstream model. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. Association for Computational Linguistics, Online, 244–255. https://doi.org/10.18653/v1/2021.findings-acl.21

Huggingface. 2021. Oscar Dataset Huggingface. Retrieved March 16, 2022 from https://huggingface.co/datasets/oscar.

Hour Kaing, Chenchen Ding, Masao Utiyama, Eiichiro Sumita, Sethserey Sam, Sopheap Seng, Katsuhito Sudoh, and Satoshi Nakamura. 2021. Towards tokenization and part-of-speech tagging for Khmer: Data and discussion. *ACM Transactions on Asian and Low-Resource Language Information Processing* 20, 6 (2021). https://doi.org/10.1145/3464378

J. S. Kikstra, P. Waidelich, J. Rising, D. Yumashev, C. Hope, and C. M. Brierley. 2021. The social cost of carbon dioxide under climate-economy feedbacks and temperature variability. *Environmental Research Letters* 16, 9 (2021), 094037.

Taku Kudo. 2018. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Melbourne, Australia, 66–75. https://doi.org/10.18653/v1/P18-1007

Taku Kudo and John Richardson. 2018. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Brussels, Belgium, 66–71. https://doi.org/10.18653/v1/D18-2012

Seth Kulick. 2011. Exploiting separation of closed-class categories for Arabic tokenization and part-of-speech tagging. *ACM Transactions on Asian Language Information Processing* 10, 1 (2011). https://doi.org/10.1145/1929908.1929912

Yachao Li, Jing Jiang, Jia Yangji, and Ning Ma. 2021. Finding better subwords for Tibetan neural machine translation. *ACM Transactions on Asian and Low-Resource Language Information Processing* 20, 2 (2021). https://doi.org/10.1145/3448216

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).

Loodos. 2020. Retrieved March 16, 2022 from Turkish Language Models. https://github.com/Loodos/turkish-language-models.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *7th International Conference on Learning Representations, (ICLR'19)*, New Orleans, LA.

Lalita Lowphansirikul, Charin Polpanumas, Nawat Jantrakulchai, and Sarana Nutanong. 2021. WangchanBERTa: Pretraining transformer-based Thai language models. *arXiv preprint arXiv:2101.09635* (2021).

Alejandro Metke Jimenez, Kerry Raymond, and Ian MacColl. 2011. Information extraction from web services: A comparison of tokenisation algorithms. In *Proceedings of the 2nd International Workshop on Software Knowledge 2011, in conjunction with 3rd International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management*. Scitepress, 12–23.

Tomás Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. In *1st International Conference on Learning Representations (ICLR'13), Workshop Track Proceedings*. Scottsdale, AZ.

Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, et al. 2019. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency* (Atlanta, GA) *(FAT*'19)*. ACM, New York, NY, 220–229. https://doi.org/10.1145/3287560.3287596

Anmol Nayak, Hariprasad Timmapathini, Karthikeyan Ponnalagu, and Vijendran Gopalan Venkoparao. 2020. Domain adaptation challenges of BERT in tokenization and sub-word representations of out-of-vocabulary words. In *Proceedings of the 1st Workshop on Insights from Negative Results in NLP*. Association for Computational Linguistics, Online, 1–5. https://doi.org/10.18653/v1/2020.insights-1.1

Pedro Javier Ortiz Suárez, Benoit Sagot, and Laurent Romary. 2019. Asynchronous pipelines for processing huge corpora on medium to low resource infrastructures. In *7th Workshop on the Challenges in the Management of Large Corpora (CMLC-7)*. Leibniz-Institut für Deutsche Sprache, Cardiff, UK, 9–16. https://doi.org/10.14618/ids-pub-9021

Chanjun Park, Sugyeong Eo, Hyeonseok Moon, and Heuiseok Lim. 2021. Should we find another model?: Improving neural machine translation performance with ONE-piece tokenization method without model modification. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Industry Papers*. Association for Computational Linguistics, Online, 97–104. https://doi.org/10.18653/v1/2021.naacl-industry.13

Kyubyong Park, Joohong Lee, Seongbo Jang, and Dawoon Jung. 2020. An empirical study of tokenization strategies for various Korean NLP tasks. In *Proceedings of the 1st Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 10th International Joint Conference on Natural Language Processing*. Association for Computational Linguistics, Suzhou, China, 133–142.

Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP'14)*. Association for Computational Linguistics, Doha, Qatar, 1532–1543. https://doi.org/10.3115/v1/D14-1162

Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. 2020. BPE-Dropout: Simple and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 1882–1892. https://doi.org/10.18653/v1/2020.acl-main.170

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI Blog* 1, 8 (2019), 9.

Lance Ramshaw and Mitch Marcus. 1995. Text chunking using transformation-based learning. In *3rd Workshop on Very Large Corpora*. https://aclanthology.org/W95-0107.

Ministry of Environment Urbanization Climate Change Republic of Turkey. 2020. Primary Energy and GHG Emissions Coefficients of Electricity. Retrieved March 16, 2022 from https://meslekihizmetler.csb.gov.tr/elektrik-enerjisinin-birincil-enerji-ve-sera-gazi-salimi-katsayilari-2021-yilindan-itibaren-kullanilmak-uzere-guncellenmistir-duyuru-411795.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019).

Mike Schuster and Kaisuke Nakajima. 2012. Japanese and Korean voice search. In *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP'12) (Kyoto, Japan)*. 5149–5152. https://doi.org/10.1109/ICASSP.2012.6289079

Stefan Schweter. 2020. *BERTurk — BERT Models for Turkish*. https://doi.org/10.5281/zenodo.3770924

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Berlin, Germany, 1715–1725. https://doi.org/10.18653/v1/P16-1162

Nakatani Shuyo. 2010. Language Detection Library for Java. Retrieved March 16, 2022 from http://code.google.com/p/language-detection/

Peter Smit, Sami Virpioja, Stig-Arne Grönroos, and Mikko Kurimo. 2014. Morfessor 2.0: Toolkit for statistical morphological segmentation. In *Proceedings of the Demonstrations at the 14th Conference of the European Chapter of the Association for Computational Linguistics*. Association for Computational Linguistics, Gothenburg, Sweden, 21–24. https://doi.org/10.3115/v1/E14-2006

Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management* (Beijing, China) *(CIKM'19)*. ACM, New York, NY, 1441–1450. https://doi.org/10.1145/3357384.3357895

Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. 2021. Charformer: Fast character transformers via gradient-based subword tokenization. *arXiv preprint arXiv:2106.12672* (2021).

Meltem Tokgoz, Fatmanur Turhan, Necva Bolucu, and Burcu Can. 2021. Tuning language representation models for classification of Turkish news. In *2021 International Symposium on Electrical, Electronics and Information Engineering*. 402–407.

Cagri Toraman, Fazli Can, and Seyit Koçberber. 2011. Developing a text categorization template for Turkish news portals. In *2011 International Symposium on Innovations in Intelligent Systems and Applications*. 379–383. https://doi.org/10.1109/INISTA.2011.5946096

Cagri Toraman, Furkan Şahinuç, and Eyup Halit Yilmaz. 2022. Large-scale hate speech detection with cross-domain transfer. In *Proceedings of the 13th Conference on Language Resources and Evaluation (LREC'22) (Marseille, France)*. 2215–2225.

Gökhan Tür, Dilek Hakkani-Tür, and Kemal Oflazer. 2003. A statistical information extraction system for Turkish. *Natural Language Engineering* 9, 2 (2003), 181–210. https://doi.org/10.1017/S135132490200284X

Ahmet Üstün and Burcu Can. 2016. Unsupervised morphological segmentation using neural word embeddings. In *International Conference on Statistical Language and Speech Processing*. Springer, 43–53.

Ahmet Üstün, Murathan Kurfalı, and Burcu Can. 2018. Characters or morphemes: How to represent words?. In *Proceedings of The 3rd Workshop on Representation Learning for NLP (Melbourne, Australia)*. 144–153.

Mihaela Alexandra Vasiu and Rodica Potolea. 2020. Enhancing tokenization by embedding Romanian language specific morphology. In *IEEE 16th International Conference on Intelligent Computer Communication and Processing (ICCP'20)*. 243–250. https://doi.org/10.1109/ICCP51029.2020.9266140

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems (CA, USA)*. 5998–6008.

Jingjing Xu, Hao Zhou, Chun Gan, Zaixiang Zheng, and Lei Li. 2021. Vocabulary learning via optimal transport for neural machine translation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, Online, 7361–7373. https://doi.org/10.18653/v1/2021.acl-long.571

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, et al. 2021. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *arXiv preprint arXiv:2105.13626* (2021).

Andrew Yates, Rodrigo Nogueira, and Jimmy Lin. 2021. Pretrained transformers for text ranking: BERT and beyond. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining (WSDM) (Virtual Event, Israel)*. 1154–1156.

Reyyan Yeniterzi. 2011. Exploiting morphology in Turkish named entity recognition system. In *Proceedings of the ACL 2011 Student Session*, Association for Computational Linguistics, Portland, OR, 105–110.

Veysel Yücesoy and Aykut Koç. 2019. Co-occurrence weight selection in generation of word embeddings for low resource languages. *ACM Transactions on Asian and Low-Resource Language Information Processing (TALLIP)* 18, 3 (2019), 1–18.

Wenbo Zhang, Xiao Li, Yating Yang, and Rui Dong. 2021b. Pre-training on mixed data for low-resource neural machine translation. *Information* 12, 3 (2021), 133.

Xinsong Zhang, Pengshuai Li, and Hang Li. 2021a. AMBERT: A pre-trained language model with multi-grained tokenization. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. Association for Computational Linguistics, 421–435. https://doi.org/10.18653/v1/2021.findings-acl.37

Received 20 April 2022; revised 21 September 2022; accepted 23 December 2022